University of Applied Sciences and Arts Northwestern Switzerland
School of Business

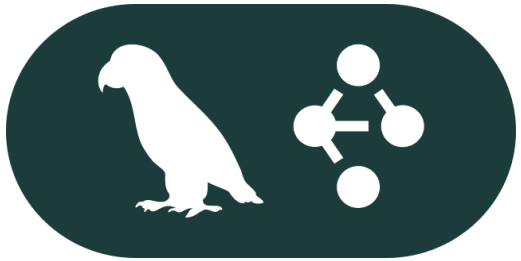
member of
swissuniversities

AACSB
ACCREDITED

Generative KI und Agentensysteme

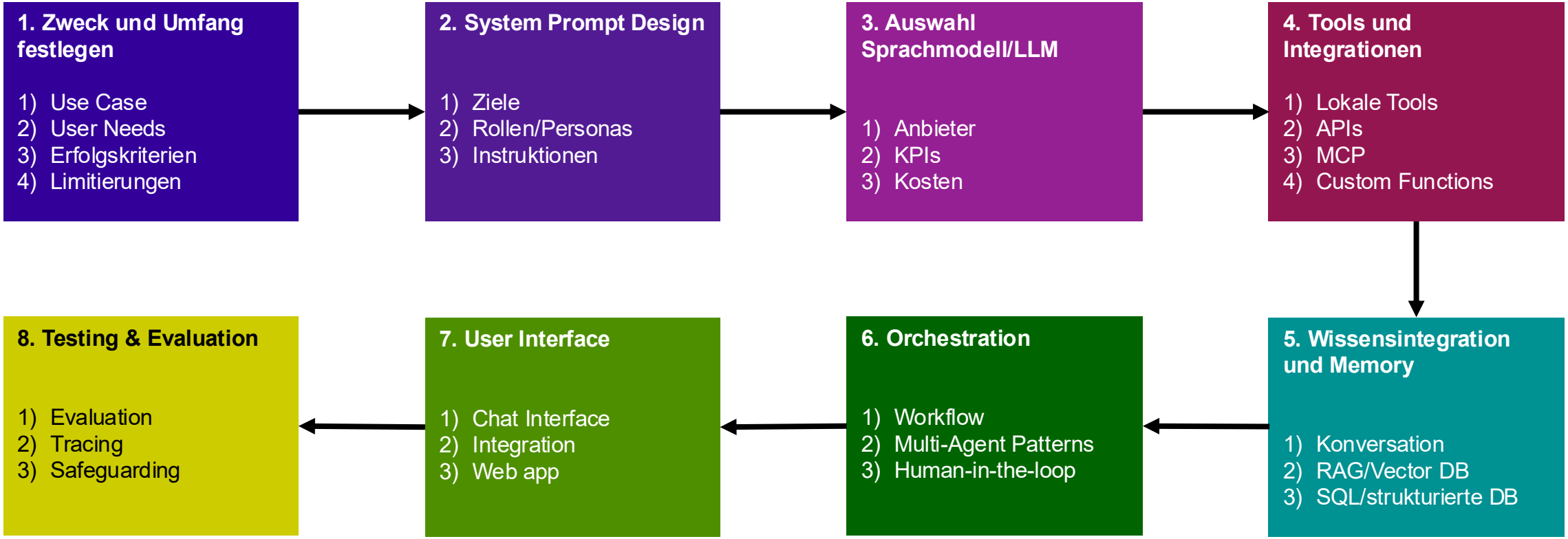
SW5

LangGraph Graph, Nodes und State



Sandro Schwander

Vom Chatbot zum Agenten: Was in welcher Woche dazu kommt



Ziel und Inhalte

Was ihr heute versteht

- Warum ein Graph und nicht eine Kette
- Knoten, Kanten und State
- Wie ein System-Prompt aufgebaut ist
- Warum Agenten strukturierte Ausgaben brauchen
- Was ein lokales Werkzeug ist und wer es ausführt

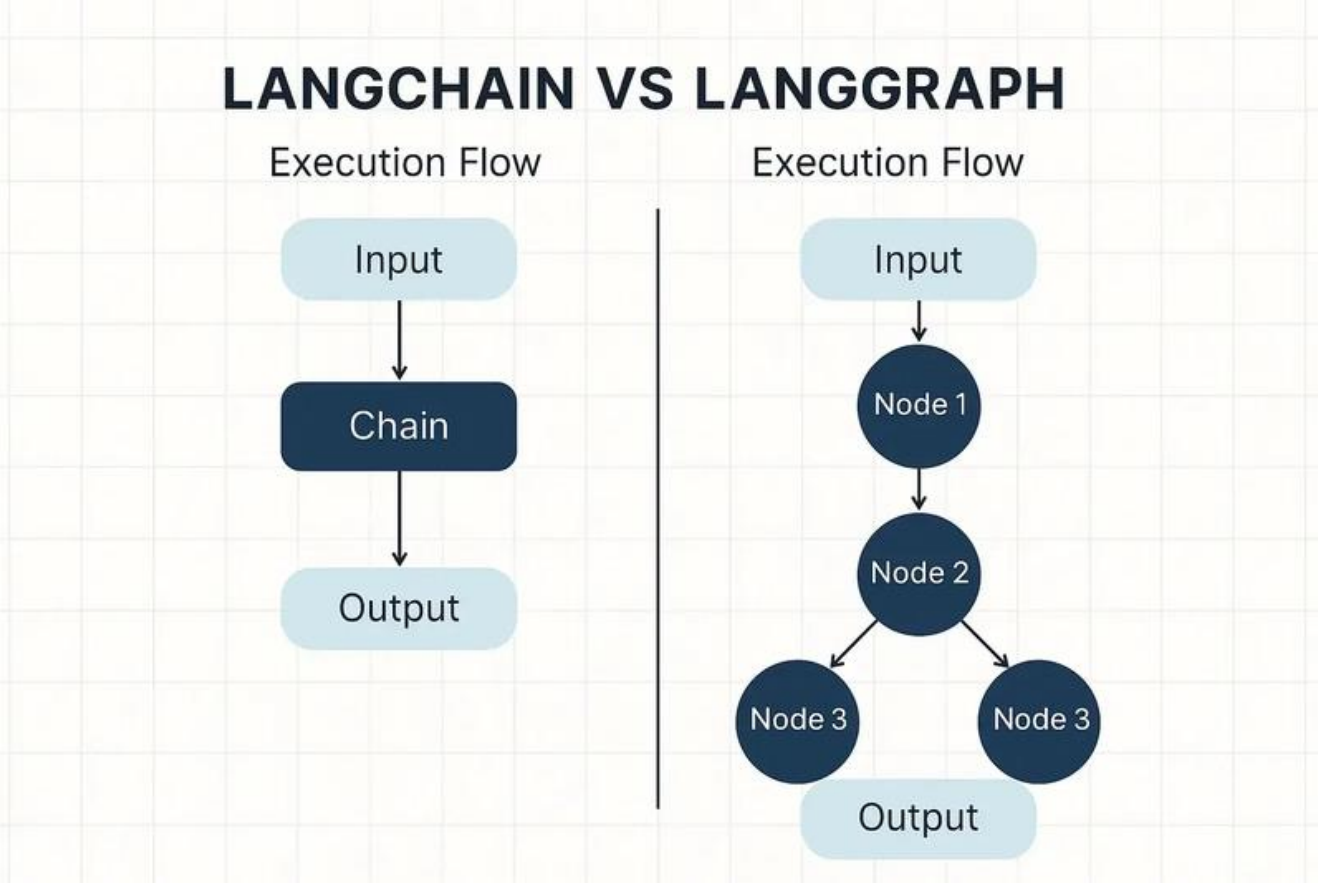
Was ihr heute tut

- Einen System-Prompt für den eigenen Use-Case schreiben
- Einen Graphen mit Klassifikation bauen und ausführen
- Ein eigenes Werkzeug schreiben und benutzen
- Tracing einschalten und einen Trace lesen

SW5 im Modulteil

- Baustein 2, 4 und 8: System Prompt Design, interne Tools, Tracing
- Woche 4: Zweck, Umfang und Modellauswahl
- Woche 6: Tools und MCP

Exkurs: LangChain vs. LangGraph



LangChain:

- lineare Abläufe konzipiert – wie Pipelines.

z.B.: Einen Chatbot, der Fragen beantwortet, ein Tool, das Google durchsucht und Ergebnisse zusammenfasst, PDF- oder CSV-Q&A-Bot

LangGraph:

- neueres Framework, das auf LangChain aufbaut. Wenn der Ablauf nicht nur linear ist.

LangGraph ist für Apps konzipiert, die Folgendes benötigen: Entscheidungen, Schleifen, Bedingte Logik, Umleitung, Zustandsverfolgung

https://dev.to/brains_behind_bots/langchain-vs-langgraph-understand-the-difference-before-you-pick-one-3hm5

LangChain als Komponentenbibliothek innerhalb von LangGraph

LangGraph – der Ablauf

- StateGraph, Knoten, Kanten, State und seine Aktualisierung compile und invoke
- Später: Verzweigungen, Schleifen, Persistenz, Unterbrechungen

LangChain – die Bauteile

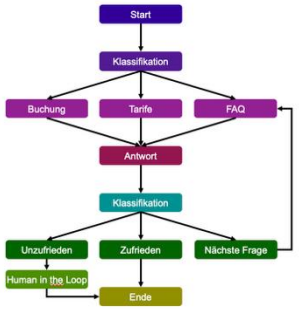
- ChatOpenAI und andere Modellanbindungen
- Document Loader und Text Splitter
- Embeddings, Vektorspeicher, Retriever
- Tool-Definitionen

Im Modul benutzen wir beides zusammen
LangGraph für die Orchestrierung, LangChain als Komponentenbibliothek. Aber: keine LCEL-Ketten

Repetition: Thinking in LangGraph – fünf Schritte zum Graphen

1. Workflow zerlegen
Welche Schritte braucht der Ablauf?

Jeder Schritt wird später ein Knoten.



2. Schritte einordnen
Was tut jeder Schritt?

LLM, Daten, Aktion oder Nutzereingabe?

- Braucht es ein Tool (call)?
- Braucht es eine Wissens-Datenbank?
- Falls ja, strukturiert? Retrieval Augmented Generation (RAG)?
- Welche Orchestration braucht es?
- Braucht es zusätzlichen Input der Nutzer*in?

3. State entwerfen
Was muss zwischen den Schritten fliessen?

Standardisierte Rohdaten, nicht fertigen (Fliess-)Text.

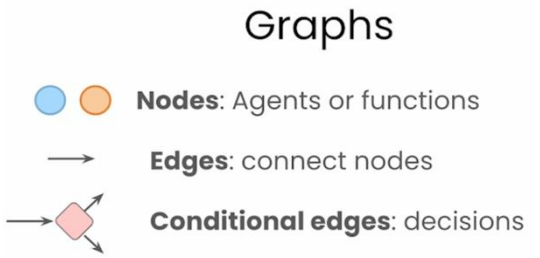
```
class EmailAgentState(TypedDict):
    # Raw email data
    email_content: str
    sender_email: str
    email_id: str

    # Classification result
    classification: EmailClassification | None

    # Raw search/API results
    search_results: list[str] | None  # List of raw document chunks
    customer_history: dict | None  # Raw customer data from CRM

    # Generated content
    draft_response: str | None
    messages: list[str] | None
```

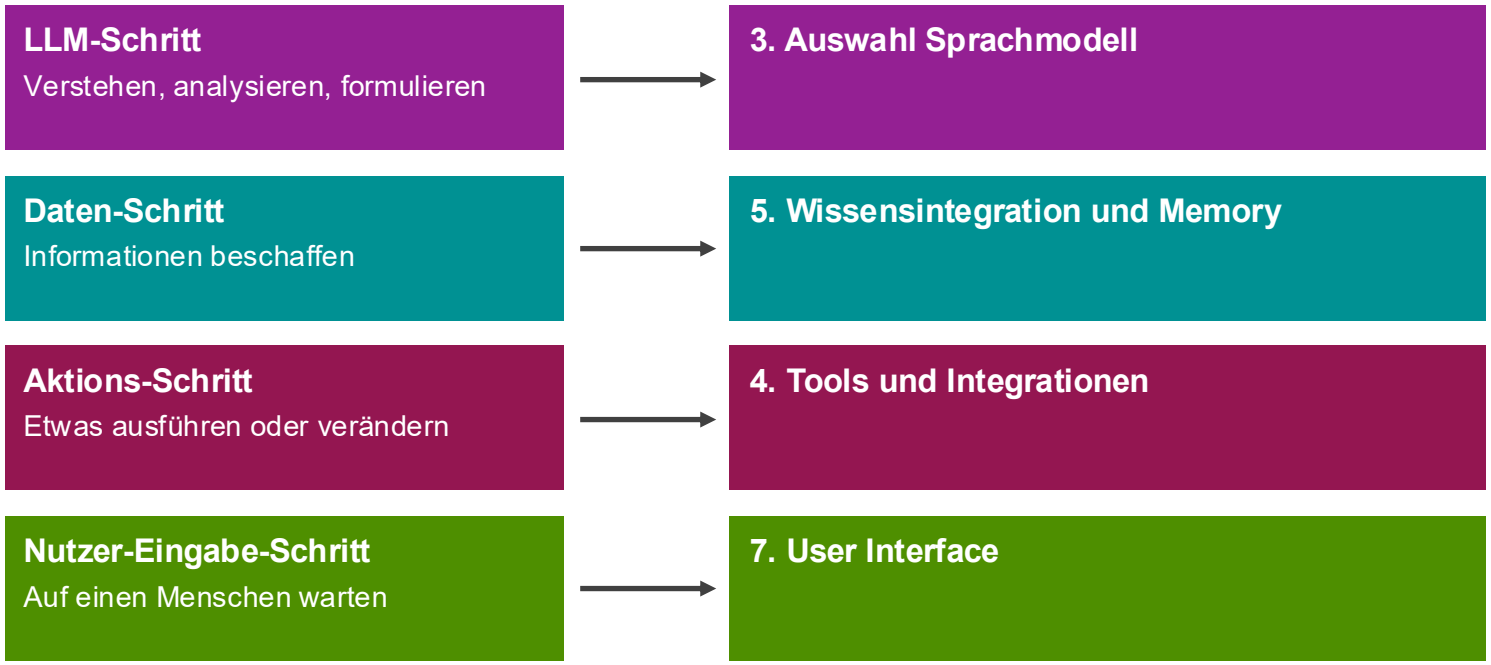
4. Knoten bauen
Jeder Knoten ist eine Funktion: State rein, Aktualisierung raus.



5. Verdrahten
Knoten mit Kanten verbinden, compile, invoke.

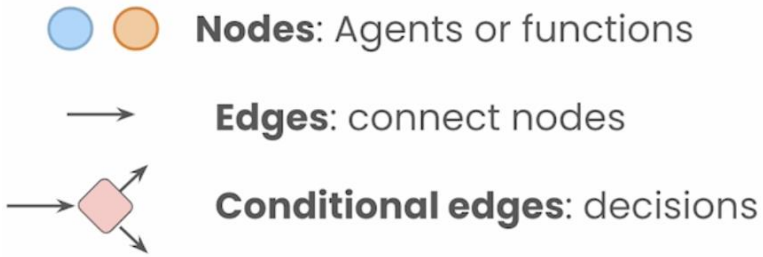
LangChain. (o. J.). Thinking in LangGraph. Abgerufen am 5. Oktober 2026, von https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

Schrittypen und Bausteine: die beiden Modelle zusammen



LangChain. (o. J.). Thinking in LangGraph,, https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

Repetition: Graph, Knoten und Kanten: die Bausteine





Node
Ein Arbeitsschritt.
Eine Funktion: sie bekommt den State und gibt eine Aktualisierung zurück.

builder.add_node("suchen", suchen)

Edges
Eine Verbindung.
Sie legt fest, was als Nächstes kommt, und tut selbst nichts.

builder.add_edge("suchen", "antworten")

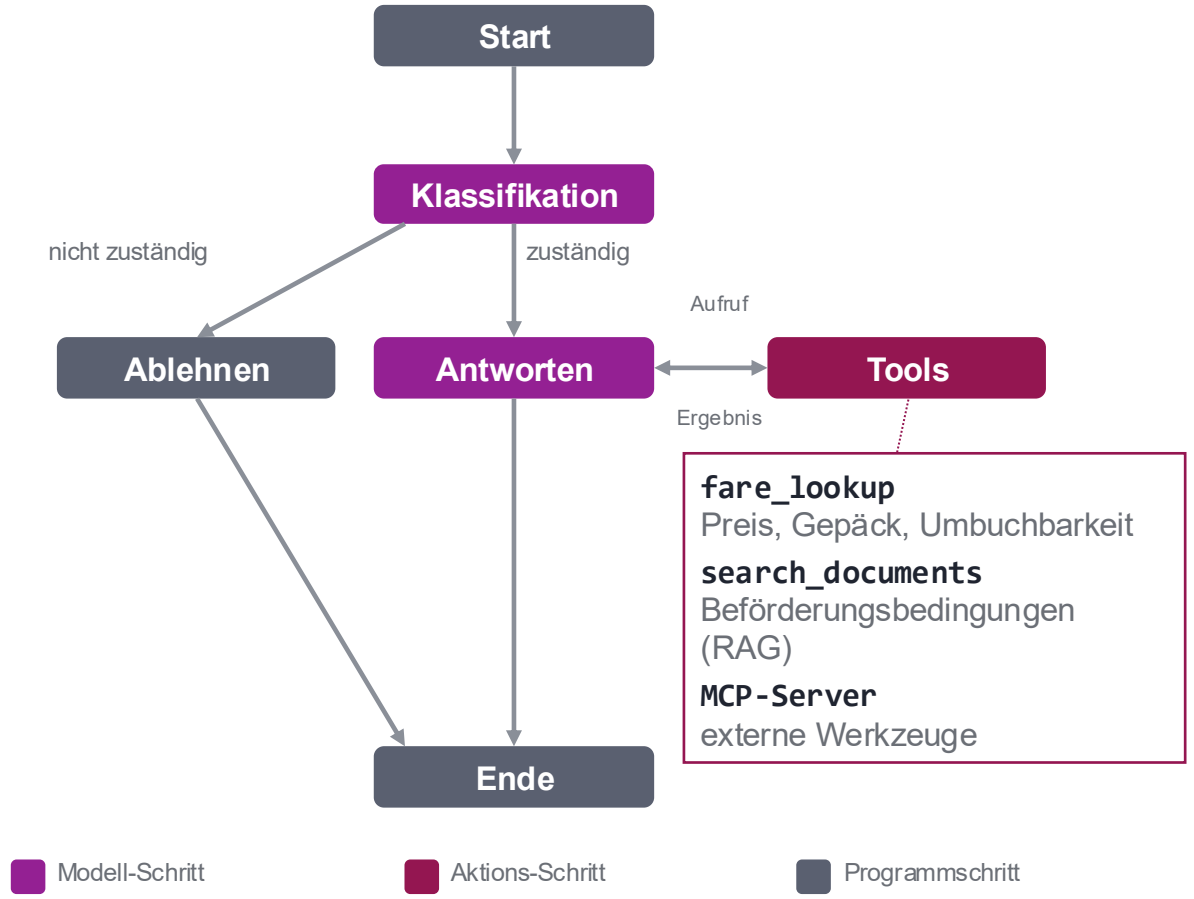
Conditional Edges

eine Funktion entscheidet zur Laufzeit, wohin es weitergeht. Erst damit entstehen Verzweigungen und Schleifen.

LangChain. (o. J.). Graph API. Abgerufen am 5. Oktober 2026, von https://docs.langchain.com/oss/python/langgraph/graph-api

LangGraph Schritt 1: Arbeitsablauf in einzelnen Schritten zerlegen

- Wo wird ein Entscheid gefällt?
- Wann braucht es allenfalls mehrere Iterationen?
- Was ist das Endprodukt?
- Welche separaten Use-Cases gibt es?



https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

State: was der Graph mitführt

Der State ist das, was zwischen den Knoten fliesst

```
from typing import TypedDict, Annotated
from langgraph.graph import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
```

Drei Fragen zum Entwurf

- Muss dieser Wert über mehrere Schritte hinweg erhalten bleiben?
- Oder lässt er sich aus etwas anderem berechnen?
- Speichere ich Rohdaten oder fertig formatierten Text?

Faustregel

Im State stehen Rohdaten (Fakten), nicht formatierter Text.
So kann jeder Knoten dieselbe Information anders darstellen.

Gut

tarif: "Economy Light"
gepaeck_kg: 8

Schlecht

text: "Im Tarif Economy Light sind 8 kg Handgepäck enthalten."

Details zu State: https://docs.langchain.com/oss/python/langgraph/use-graph-api

LangChain. (o. J.). Thinking in LangGraph. Abgerufen am 5. Oktober 2026, von https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

Ein minimaler Graph im Code

```
from typing import TypedDict, Annotated
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END, add_messages

llm = ChatOpenAI(model=MODEL, base_url=BASE_URL, api_key=API_KEY)

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

def antworten(state: AgentState):
    antwort = llm.invoke(state["messages"])
    return {"messages": [antwort]}

builder = StateGraph(AgentState)
builder.add_node("antworten", antworten)
builder.add_edge(START, "antworten")
builder.add_edge("antworten", END)
graph = builder.compile()
```

Die vier Zeilen, auf die es ankommt

- **AgentState:** legt fest, was mitfliesst.
- **Antworten:**ist der einzige Knoten: State rein, Aktualisierung raus.
- **add_edge:** verbindet START, Knoten und END.
- **Compile:** macht aus der Beschreibung etwas Ausführbares.

Ausführung: compile und invoke

Beschreiben

```
builder = StateGraph(AgentState)
builder.add_node(...)
builder.add_edge(...)
```

Hier läuft noch nichts.
Es entsteht nur ein Objekt, das den Ablauf beschreibt.

Ausführen

```
graph = builder.compile()

graph.invoke({"messages": [("user", frage)]})
```

Erst compile macht den Graphen ausführbar.
invoke startet einen Durchlauf und gibt den End-State zurück.

Warum diese Trennung wichtig ist

Weil die Runtime den Ablauf als Datenstruktur kennt und nicht nur als Programmablauf, kann sie ihn zeichnen, Schritt für Schritt streamen, nach jedem Knoten speichern, anhalten und später fortsetzen. Das ist die Grundlage für Woche 8.

System-Prompt: Ziel und Aufgabe

Ziel und Aufgabe

- **Was soll der Agent erreichen?**
- **Rolle und Persona**
- **Instruktion zu Antwort und Format**
- **Allenfalls Guardrails (SW9)**

Beispiel SWISS-Agent

```
SYSTEM_PROMPT = """Du bist ein Assistent, der Fragen zu Tarifen und Gepäck bei
SWISS International Air Lines beantwortet.

Fakten:
- Schlage Tarifkonditionen und Gebühren immer mit dem Werkzeug fare_lookup nach.
- Steht etwas nicht in den Daten, sage das. Erfinde keine Gepäckregeln,
  Gebühren, Fristen oder Kontaktdaten wie Telefonnummern.
- Fehlt eine Angabe, etwa der Tarif, frage nach, statt zu raten.

Antwortstil:
- Sprich die Person mit «Sie» an und antworte in der Sprache der Frage.
- Höchstens vier Sätze oder eine kurze Liste, keine Überschriften.
- Nenne bei Gebühren immer den Tarif, auf den sie sich beziehen.

Kennzeichnung:
- Du bist ein Prototyp aus einer Lehrveranstaltung der FHNW und kein offizieller
  Dienst von SWISS. Sage das, wenn jemand dich für den offiziellen Kundendienst
hält."""
```

Details zum System-Prompt in LangChain/LangGraph: https://docs.langchain.com/oss/python/langchain/messages#system-message

Schwacher und starker System-Prompt im Vergleich

Schwach

```
Du bist ein hilfreicher Assistent für
eine Fluggesellschaft. Antworte gut
und freundlich.
```

Was passiert

Das Modell erfindet Gepäckregeln, wenn es keine findet.

Die Antwortlänge schwankt stark.

Es sagt Dinge zu, die es nicht zusagen darf.

Es bleibt unklar, woher die Information stammt.

Stark

```
Du beantwortest Fragen zu Tarifen von
SWISS nur aus den Dokumenten. Höchstens
fünf Sätze. Quelle nennen. Keine Zusagen.
Fehlt die Information: sag es.
```

Was sich ändert

Aufgabe, Grenze, Format und Verhalten im Zweifel sind festgelegt.

Die Antworten werden vergleichbar – und damit prüfbar.

Dieselben vier Elemente wie auf den letzten Folien: Ziel, Rolle, Instruktionen, Leitplanken.

> Aufgabe

- Geht in eurer Gruppe zusammen
- Definiert einen sinnvollen System Prompt, den ihr heute verwenden könnt (V0.5)

Zeit: 20 Min.

Strukturierte Ausgaben: warum Agenten sie brauchen

Freier Text

```
"Im Tarif Economy Light sind 8 kg
 Handgepäck enthalten, Aufgabegepäck
 kostet extra."
```

Problem

- Ein weiterer Schritt müsste diesen Satz wieder auseinandernehmen.
- Formulierung und Reihenfolge ändern sich bei jedem Aufruf.
- Prüfen lässt sich so nur schwer etwas.

Strukturierte Ausgabe

```
{"tarif": "Economy Light",
 "handgepaeck_kg": 8,
 "aufgabegepaeck": false}
```

Vorteil

Der nächste Knoten kann direkt damit rechnen.

Fehlende Felder fallen sofort auf.

Eine Evaluation kann Feld für Feld vergleichen – Woche 9.

Für Agenten ist das der Normalfall: Was ein Knoten an den nächsten gibt, sollte eine Struktur sein, kein Fliesstext.

Schema definieren, validieren

```python
from pydantic import BaseModel, Field

class Tarifauskunft(BaseModel):
    tarif: str = Field(description="Name des Tarifs")
    handgepaeck_kg: int
    aufgabegepaeck: bool
    antwort: str = Field(description="Antwort in höchstens fünf Sätzen")

llm_strukturiert = llm.with_structured_output(Tarifauskunft)

ergebnis = llm_strukturiert.invoke(frage)
ergebnis.handgepaeck_kg   # 8
```

Schema definieren

Ein Modell mit Feldern und Typen. Die Beschreibungen sind Teil des Prompts.

Validieren

Passt die Antwort nicht zum Schema, gibt es einen Fehler statt stillem Unsinn.

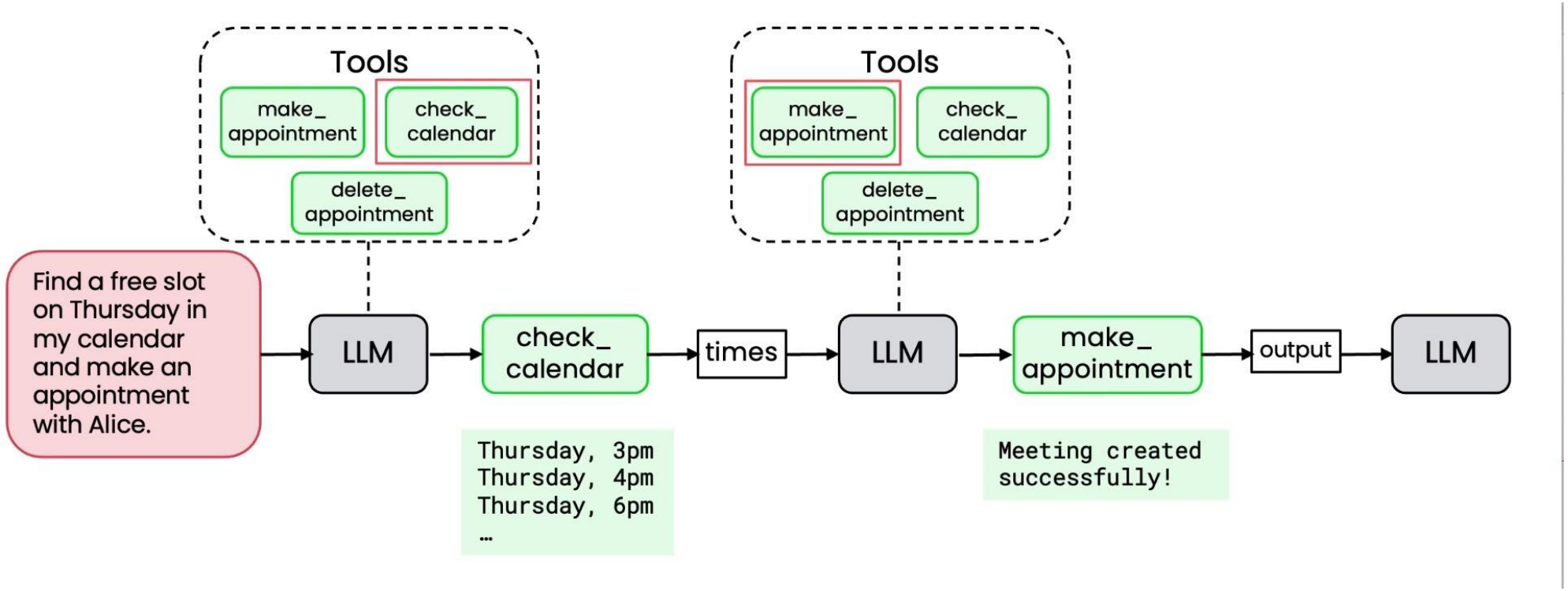
LangChain. (o. J.). Structured output. Abgerufen am 5. Oktober 2026, von https://docs.langchain.com/oss/python/langchain/structured-output

Demo 1

- Prompt
- Strukturierte Ausgaben

Tools – lokales Tool

Tool Use – Funktionen/APIs aufrufen



DeepLearning.ai - Andrew Ng: https://learn.deeplearning.ai/courses/agentic-ai/

Anatomie eines Tools

```python
from langchain_core.tools import tool

@tool
def fare_lookup(fare: str) -> str:
    """Schlägt Konditionen und Nebenkosten eines SWISS-Tarifs nach.

    Liefert Handgepäck, Freigepäck, Gebühr für ein zusätzliches Gepäckstück,
    Umbuchungsgebühr und Sitzplatzwahl.
    Mögliche Tarife: Economy Light, Economy Classic, Economy Flex.
    """
    data = FARES.get(fare.strip().lower())
    if data is None:
        return f"Tarif '{fare}' ist nicht in der Tabelle. Bekannt sind: {',
'.join(FARES)}."
    return ", ".join(f"{k}: {v}" for k, v in data.items())
```

Name
Woran das Modell das Tool erkennt.

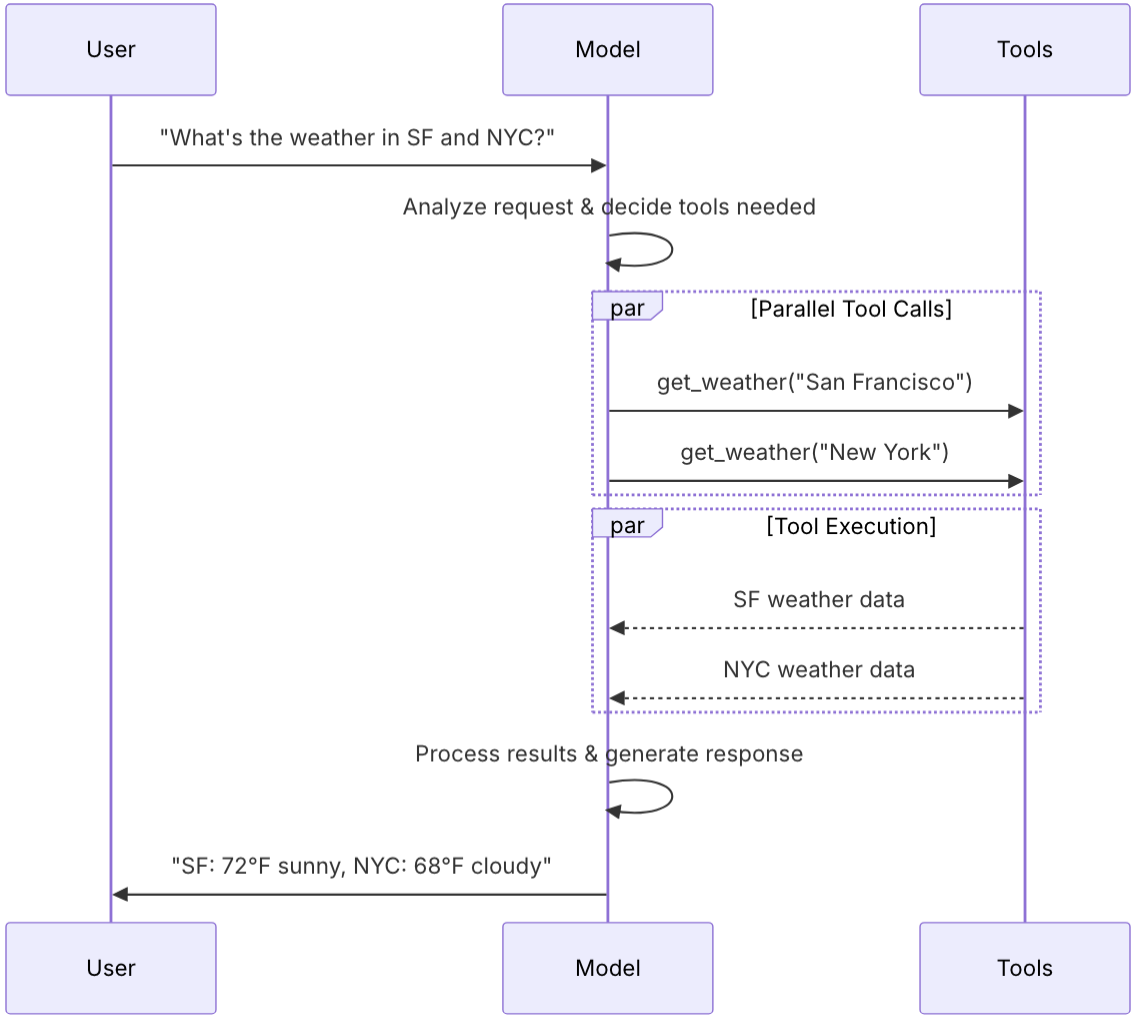
Beschreibung
Der Docstring. Er geht als Text ans Modell.

Parameter mit Typen
Daraus entsteht das Schema für den Aufruf.

Rückgabewert
Geht als Nachricht zurück ins Gespräch.

Details zu Tools: https://docs.langchain.com/oss/python/langchain/tools

LangChain. (o. J.). Tools. Abgerufen am 5. Oktober 2026, von https://docs.langchain.com/oss/python/langchain/tools



https://docs.langchain.com/oss/python/langchain/models#tool-calling

Der Ablauf eines Tool-Calls

1. Frage

Die Nutzerfrage geht mit allen
Tool-Definitionen ans Modell.

▸

2. Entscheidung

Das Modell antwortet mit einem
tool_call statt mit Text.

▸

3. Ausführung

Euer Code führt die Funktion aus.
Hier passiert die eigentliche Arbeit.

▸

4. Rückgabe

Das Ergebnis geht als Nachricht
vom Typ "tool" zurück ins Gespräch.

▸

5. Antwort

Das Modell formuliert daraus
die Antwort für die Nutzerin.

Schritt 3 ist der entscheidende: Das Modell führt nichts aus. Es schlägt vor, euer Programm entscheidet und handelt.

Details zu Tool-Calling: https://docs.langchain.com/oss/python/langchain/models#tool-calling

Tools einbinden: binden, ausführen, Rückkante

```
from langchain_core.messages import ToolMessage

tools = [fare_lookup]
llm_mit_tools = llm.bind_tools(tools)
nach_name = {t.name: t for t in tools}

def answer(state):                      # Modell-Schritt
    return {"messages": [llm_mit_tools.invoke(state["messages"])]}

def call_tools(state):                  # Aktions-Schritt
    raus = []
    for aufruf in state["messages"][-1].tool_calls:
        ergebnis = nach_name[aufruf["name"]].invoke(aufruf["args"])
        raus.append(ToolMessage(content=str(ergebnis),
                                tool_call_id=aufruf["id"]))
    return {"messages": raus}

def route_after_answer(state):          # bedingte Kante
    return "tools" if state["messages"][-1].tool_calls else END

builder.add_node("answer", answer)
builder.add_node("tools", call_tools)
builder.add_conditional_edges("answer", route_after_answer,
                              {"tools": "tools", END: END})
builder.add_edge("tools", "answer")     # Rückkante
```

bind_tools

Das Modell bekommt Name, Beschreibung und Parameterschema jedes Werkzeugs mitgeliefert. Ausführen tut es nichts.

call_tools

Ein gewöhnlicher Knoten: Er liest die tool_calls, ruft die Funktion auf und legt das Ergebnis als ToolMessage in den State.

Die Rückkante

route_after_answer schickt weiter zu «tools» oder zu END, «tools» → «answer» schliesst den Kreis.

Fertig gibt es dasselbe als ToolNode und tools_condition. Wir bauen es von Hand, damit der Ablauf sichtbar bleibt.

Details zur Tool-Schleife im Graphen: https://docs.langchain.com/oss/python/langgraph/quickstart
Conditions: https://reference.langchain.com/python/langgraph.prebuilt/tool_node/tools_condition
Tool Node: https://reference.langchain.com/python/langgraph.prebuilt/tool_node/ToolNode

LangChain. (o. J.). Graph API – Tool calling. Abgerufen am 5. Oktober 2026, von https://docs.langchain.com/oss/python/langgraph/graph-api

Tracing & Debugging

Tracing & Debugging in LangChain

	Verbose Mode	Debug Mode	LangSmith Tracing
Free	✅	✅	✅
UI	❌	❌	✅
Persisted	❌	❌	✅
See all events	❌	✅	✅
See "important" events	✅	❌	✅
Runs Locally	✅	✅	❌

https://python.langchain.com/docs/how_to/debugging/

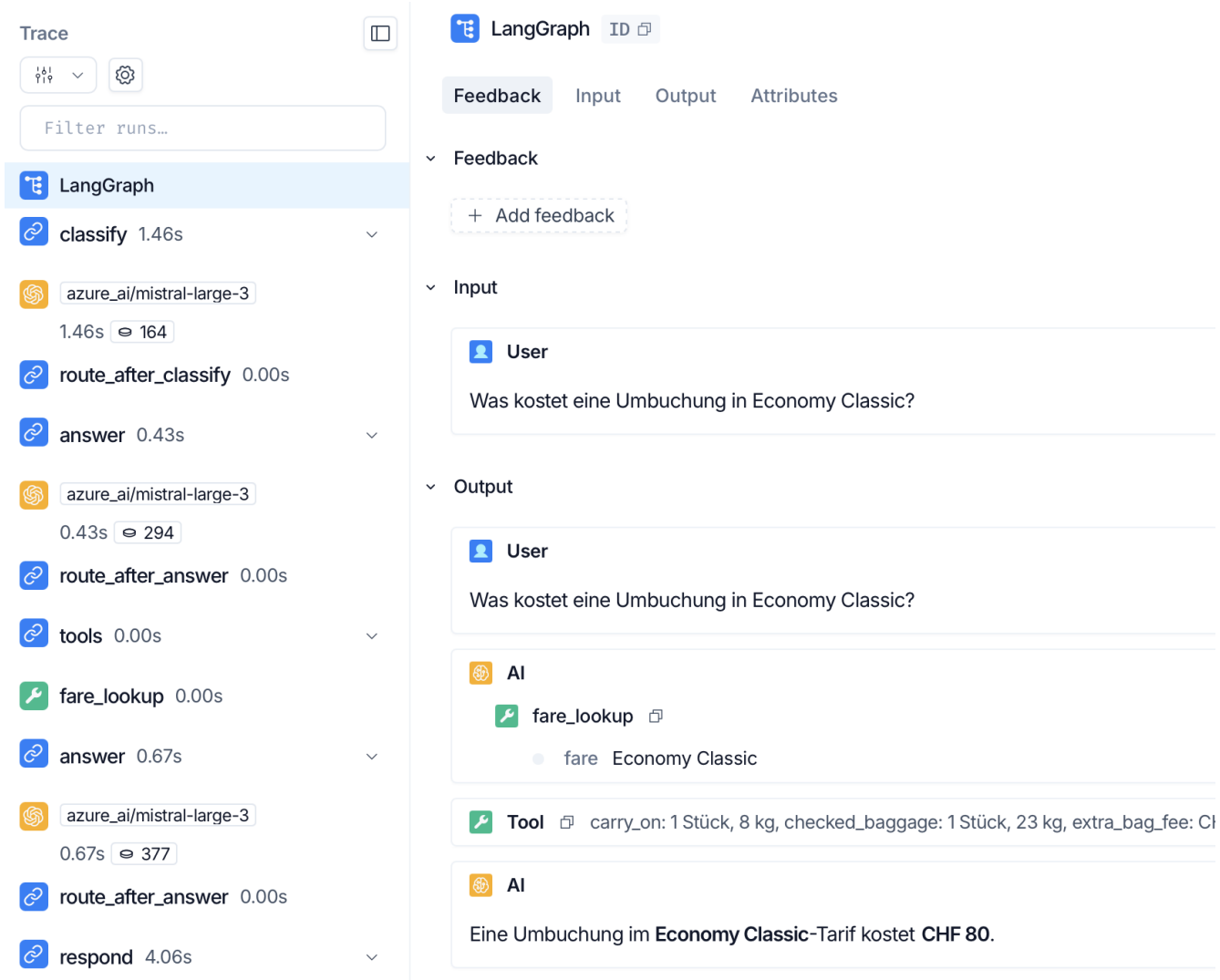
LangSmith und LangFuse

- LangSmith:
  - LangSmith ist eine einheitliche Plattform für Beobachtbarkeit und Auswertung, auf der Teams die Leistung von KI-Anwendungen debuggen, testen und überwachen können – unabhängig davon, ob sie mit LangChain erstellt wurden oder nicht.
  - Direkt integriert mit LangChain, free-tier für developer
- LangFuse: OpenSource tracing, das auch mit LangChain funktioniert, aber von LangChain unabhängig ist.
  - Funktioniert mit verschiedensten Anbietern, z.b. auch Llama-Index, Haystack, Flowise oder crewAI

https://www.langchain.com/langsmith,

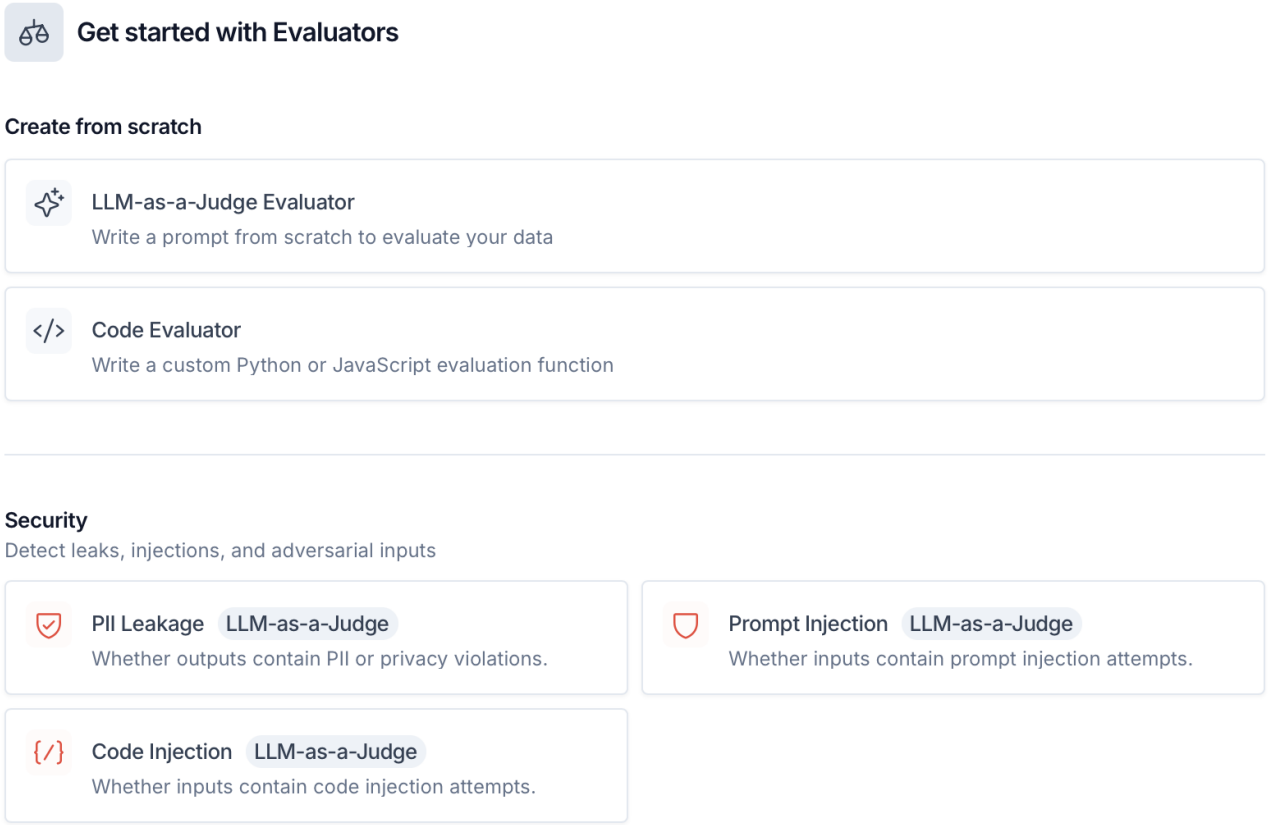
Trace Beispiel

- Kann von Schritt zu schritt durchgehen
- Sieht was wann geschah, auch Probleme oder Fehler



Evaluators

- Man kann auch noch Evaluatoren hinzufügen
- Per Code oder LLM-as-a-Judge



Graph API und Functional API

Graph API – diese verwenden wir

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(AgentState)
builder.add_node("antwort", antwort)
builder.add_edge(START, "antwort")
builder.add_edge("antwort", END)
graph = builder.compile()
```

Erkennungsmerkmale

StateGraph, add_node, add_edge, compile

Der Ablauf ist ein Objekt, bevor er läuft

Graph lässt sich zeichnen, anhalten, fortsetzen

Functional API – nur zur Einordnung

```
from langgraph.func import entrypoint, task

@task
def antwort(frage): ...

@entrypoint()
def app(frage): return antwort(frage).result()
```

Erkennungsmerkmale

@entrypoint und @task

Gewöhnlicher Kontrollfluss, kein Graph-Objekt

Gleiche Runtime, gleiche Fähigkeiten

LangChain. (o. J.). Choosing between Graph and Functional APIs. Abgerufen am 5. Oktober 2026, von https://docs.langchain.com/oss/python/langgraph/choosing-apis

Demo 2

- Tool „fare_lookup“
- Den Graphen zusammenbauen
- Tracing

> Aufgabe Gruppenarbeit

1. Entwickelt eine erste Version eures Agenten
    1. Prompt Templates
    2. State Design
    3. (Conditional) Edges
2. Dies kann eine vereinfachte Version sein, beispielsweise nur mit einem lokalen Tool (Analog den Inhalten dieser Woche)
3. Verknüpft mit LangSmith und schaut euch die Traces im Detail an.
4. Könnt ihr eure Erfolgskriterien, die ihr in der letzten Woche definiert habt, messen?